

Databázové systémy

Úvod do PL/pgSQL - 1. časť

Ing. Ján Perháč, PhD.

Katedra počítačov a informatiky
Fakulta elektrotechniky a informatiky
Technická univerzita v Košiciach



Košice, Slovensko

PL/pgSQL

Úvod

- SQL nie je *Turing complete*, t.j. nemá vyjadrovaciu silu Turingovho stroja.
- PL/pgSQL umožňuje písať imperatívny kód.
 - Riadiace konštrukcie cyklov, vetvenia, atď.
 - Definovať a spúšťať programové jednotky.
- Procedural Language extension of Structured Query Language.
 - Oracle PL/SQL,
 - Microsoft SQL Server používa Transact-SQL.
- PostgreSQL v súčasnosti podporuje niekoľko štandardných procedurálnych jazykov.
 - PL/pgSQL, PL/Tcl, PL/Perl, PL/Python, PL/Java,
 - a mnoho ďalších.
- V PostgreSQL je predvolený **PL/pgSQL**.

PL/pgSQL

Bloky

- Základnou jednotkou v akomkoľvek kóde PL/pgSQL je **Blok**.
- Celý kód PL / pgSQL sa skladá z jedného bloku alebo viacerých blokov, ktoré sa píšú sekvenčne a/alebo sú vnorené do iného bloku.
- Existujú dva druhy blokov:
 - **Anonymné bloky.**
 - Konštruované dynamicky a vykonávané iba raz používateľom.
 - **Pomenované bloky** (funkcie a storované procedúry).
 - Majú priradené meno,
 - sú uložené v databáze,
 - dajú sa vykonať opakovane,
 - môžu obsahovať parametre.

PL/pgSQL

Úvodzovky prostredníctvom \$\$

```
select 'String constant';  
select 'I"m also a string constant';  
$tag$ <string_constant> \ $tag$  
select $$I'm a string with a backslash \$$;  
select $message$I'm a string with a backslash \ $message$;
```

PL/pgSQL

Anonymné bloky - štruktúra

```
DO $$  
[ <<label>> ]  
DECLARE  
/* Declare section (optional). */  
BEGIN  
/* Executable section (required). */  
EXCEPTION  
/* Exception handling section (optional). */  
END [ label ]  
$$;
```

PL/pgSQL

Anonymné bloky - príklad

```
DO $$  
<<prvyblok>>  
DECLARE  
  counter integer;  
BEGIN  
  select count(*) into counter from mesto;  
  raise notice 'Pocet miest je %', counter;  
END prvyblok  
$$;
```

PL/pgSQL

Komentáre

- V PL/pgSQL existujú dva typy komentárov.
 - Jednoriadkový komentár:
 - - *spustí komentár, ktorý siaha až po koniec riadku.*
 - Viacriadkový komentár:
 - / * *viacriadkové komentáre* * /
- Komentovanie je nevyhnutné pre vysvetlenie, čo je ako zamýšľané a prečo to bolo urobené daným konkrétnym spôsobom.

Premenné a konštanty

Premenné

- Premenné sú využívané pre:
 - Dočasné uloženie údajov.
 - Manipulácia s uloženými hodnotami.
 - Opätovná použiteľnosť.
- Deklarované v deklaratívnej časti v rámci bloku.
- Premenné deklarované sa inicializujú na svoje predvolené hodnoty zakaždým, keď je blok volaný, nielen raz pri volaní funkcie.
- Premenné v deklaratívnej časti môžu *preťažiť* premenné s rovnakým menom deklarované vo vonkajšom bloku.
- Ak je vonkajší blok pomenovaný a label, potom jeho premenné sú dostupné v rámci vnoreného bloku pomocou:

<label>. <varname>

Premenné a konštanty

Deklarácie

- Syntax:

```
identifier [CONSTANT] datatype [NOT NULL]  
[:= | = | DEFAULT expr];
```

- Príklady:

```
DECLARE
```

```
v_birthday DATE;
```

```
v_age INT NOT NULL = 21;
```

```
v_name VARCHAR(15) := 'Homer';
```

```
v_magic CONSTANT NUMERIC := 42;
```

```
v_valid BOOLEAN DEFAULT TRUE;
```

Premenné a konštanty

SELECT INTO

- Príkaz **SELECT INTO** umožňuje výber údajov z databázy a priradenie údajov premennej.
- Príklad:

```
select select_list  
into variable_name  
from table_expression;
```

Premenné a konštanty

%TYPE

- Umožňuje deklarovať premennú podľa:
 - Definície stĺpca databázy.
 - Ďalšej predtým deklarovanej premennej.

- Syntax:

```
identifier table.column_name%TYPE;
```

- Príklad:

```
DECLARE
```

```
v_email users.email%TYPE;
```

```
v_my_email v_email%TYPE :=  
    'janko.hrasko@tuke.sk';
```

Premenné a konštanty

%ROWTYPE

- Pre uloženie celého riadku množiny výsledkov vrátenej príkazom `select into`, použijete premennú typu *riadok* (*row-type*).
- Premennú, ktorá má rovnaký údajový typ ako údajový typ riadku v tabuľke, je možné deklarovať nasledovne:

```
identifier table_name%ROWTYPE;
```

- Na prístup k prvkom premennej riadka slúži zápis `(.)`:

```
identifier.field_name
```

Premenné a konštanty

%ROWTYPE - príklad

```
DO $$
```

```
DECLARE
```

```
    typriadok mesto%rowtype;
```

```
BEGIN
```

```
    select * into typriadok from mesto where id=1;
```

```
    raise notice 'Mesto s ID % je % a ma % obyvateľov.',
```

```
    typriadok.id, typriadok.nazov, typriadok.populacia;
```

```
END
```

```
$$;
```

Premenné a konštanty

Záznam (Record)

- Záznam je typ premennej podobný ROWTYPE, ale bez preddefinovanej štruktúry.
- Skutočná štruktúra záznamu sa vytvorí pri prvom priradení premennej.
- Záznam nie je skutočný dátový typ, iba *place holder*.
- Využitie napr. pri cykloch.
- Príklad:

```
DECLARE  
    r record;
```

Premenné a konštanty

Záznam (Record) - príklad

```
DO $$
```

```
DECLARE
```

```
    zaznammiest record;
```

```
BEGIN
```

```
    for zaznammiest in select nazov, populacia from mesto loop
```

```
        raise notice 'Mesto % ma % obyvateľov.',
```

```
        zaznammiest.nazov, zaznammiest.populacia;
```

```
    end loop ;
```

```
END
```

```
$$;
```

Výstup:

```
... Mesto Uzhorod ma 115512 obyvateľov.
```

```
... Mesto Kyjev ma 2963199 obyvateľov.
```

```
... Mesto Kosice ma 238757 obyvateľov.
```

```
... Mesto Presov ma 100000 obyvateľov.
```

```
... Mesto Odesa ma 1000000 obyvateľov.
```

```
... Mesto Bardejov ma 40000 obyvateľov.
```

```
... Mesto Michalovce ma 40000 obyvateľov.
```

Premenné a konštanty

Rozsah viditeľnosti premennej (*variable scope*)

```
DO $$
```

```
DECLARE
```

```
cislo integer := 30;
```

```
BEGIN
```

```
RAISE NOTICE 'cislo tu je %', cislo; - - 30
```

```
cislo := 50;
```

```
- - vytvorenie podbloku
```

```
DECLARE
```

```
cislo integer := 80;
```

```
BEGIN
```

```
RAISE NOTICE 'cislo tu je %', cislo; - - 80
```

```
END;
```

```
RAISE NOTICE 'cislo tu je %', cislo; - - 50
```

```
END
```

```
$$;
```


Premenné a konštanty

Rozsah viditeľnosti premennej (*variable scope*)

- prístup k premenným vyššieho bloku

DO \$\$

<< hlavnyblok >>

DECLARE

cislo integer := 30;

BEGIN

RAISE NOTICE 'cislo tu je %', *cislo*; - - 30

cislo := 50;

- - vytvorenie podbloku

DECLARE

cislo integer := 80;

BEGIN

RAISE NOTICE 'cislo tu je %', *hlavnyblok.cislo*; - - 50

RAISE NOTICE 'cislo tu je %', *cislo*; - - 80

END;

RAISE NOTICE 'cislo tu je %', *cislo*; - - 50

END

\$\$;

Premenné a konštanty

Konštanty

- Na rozdiel od *premennej* nemožno hodnotu *konštanty* po jej inicializácii zmeniť.
- Prečo využívať konštanty:
 - Zvyšujú čitateľnosť a zrozumiteľnosť kódu.
 - Redukujú náročnosť údržby kódu.

- Syntax:

constant_name **constant** *data_type* := *expression*;

- Príklad:

```
predajna_cena := cena + cena * 0.2;
```

```
predajna_cena := cena + cena * DPH;
```

Správy a chybové hlásenia

Správy

- Pre vyvolanie správy slúži príkaz *raise* nasledovne:
raise level format;
- *Level* - nasleduje po *raise*. Špecifikuje závažnosť chyby. Môže nadobúdať nasledujúce:
 - debug, log, notice, info, warning, exception.
- Ak neurčíte *level*, v predvolenom nastavení použije príkaz *exception*, ktorý spôsobí chybu a zastaví aktuálnu transakciu.
- *Format* - Formát je reťazec, ktorý špecifikuje správu. Formát používa ako *place holder* symbol(y) (%), ktoré budú nahradené argumentmi.
- Počet % musí byť rovnaký ako počet argumentov, inak PostgreSQL vyvolá chybu.

Správy a chybové hlásenia

Správy - príklad

```
do $$  
begin  
  raise info 'information message %', now() ;  
  raise log 'log message %', now();  
  raise debug 'debug message %', now();  
  raise warning 'warning message %', now();  
  raise notice 'notice message %', now();  
end $$;
```

Správy a chybové hlásenia

Chybové hlásenia

- Na vyvolanie *chyby* sa využíva príkaz **exception** na mieste *level* po príkaze *raise*.
- Predvolené správanie *raise*.
- Okrem upozornenia na chybu, je možné doplniť ďalšie informácie pomocou nasledujúcej klauzuly:
using option = expression
- Kde *option* môže byť:
 - *message*,
 - *hint*,
 - *detail*,
 - *errcode*.
- Príklad:
raise exception 'duplicate email: %', email
using hint = 'check the email again';

Riadiace štruktúry

Riadiace štruktúry v PL/pgSQL

- Riadiace štruktúry v PL/pgSQL je možné rozdeliť do dvoch kategórií.
 - Podmienené príkazy (*If, case*).
 - Príkazy cyklu (*loop, while, for*).

Príkaz IF

- Príkaz **if** určuje, ktoré príkazy sa majú vykonať na základe výsledku boolovského výrazu.
- **PL/pgSQL** podporuje tri formy príkazu **if**.
 - *if then*
 - *if then else*
 - *if then elsif*

Radiace štruktúry

Príkaz **IF ... THEN ...**

- Príkaz **if** vykoná príkazy, ak je podmienka splnená. Ak je podmienka vyhodnotená ako nepravdivá, pokračuje sa vykonávaním ďalšieho príkazu po klauzule **end if**.
- *Podmienka* je boolovský výraz.
- *Príkazy* môžu byť jeden alebo viac príkazov, ktoré sa vykonajú, ak je podmienka splnená. Môže to byť akýkoľvek platný príkaz, resp. aj iný príkaz **if**.
- Keď je príkaz **if** umiestnený do iného príkazu **if**, nazýva sa to *vnorený príkaz if* (nested).
- Syntax:
if podmienka **then**
 príkazy;
end if;

Riadiace štruktúry

Príkaz **IF ... THEN ... ELSE ...**

- Príkaz **if ... then ... else ...** vykoná príkazy vo vetve *then*, ak je podmienka vyhodnotená ako *true*, v opačnom prípade vykoná príkazy vo vetve *else*.
- Syntax:
if *podmienka* **then**
 príkazy;
else
 alternatívne_príkazy;
end if;

Riadiace štruktúry

Príkaz IF ... THEN ... ELSIF ... THEN ... ELSE ...

- Príkazy *if* a *if then else* vyhodnotia jednu podmienku.
- Alternatíva pre vnorené *if then else*.
- Príkaz *if then elsif* však vyhodnotí viac podmienok.
- Ak je podmienka pravdivá, vykoná sa zodpovedajúci príkaz v danej vetve.
- Ak sa všetky podmienky vyhodnotia ako nepravdivé, príkaz *if elsif* vykoná príkazy vo vetve *else*.

- Syntax:

```
if podmienka1 then  
    príkazy1;  
if podmienka2 then  
    príkazy2;  
...  
if podmienka n then  
    príkazy n;  
else  
    alternatívne_príkazy;  
end if;
```

Riadiace štruktúry

Príkaz **IF ... THEN ... ELSIF ... THEN ... ELSE ...** - príklad

IF number = 0 **THEN**

result := 'zero';

ELSIF number > 0 **THEN**

result := 'positive';

ELSIF number < 0 **THEN**

result := 'negative';

ELSE

- -the only other possibility is that number is null

result := '**NULL**';

END IF;

Riadiace štruktúry

Príkaz **CASE**

- Okrem príkazu **if** PostgreSQL poskytuje príkazy **case**, ktoré umožňujú vykonať blok kódu na základe podmienky.
- Príkaz **case** vyberie **when section** na základe podmienky.
- Príkaz **case** má dve formy:
 - Jednoduchý Príkaz **case**.
 - Hľadaný príkaz **case**.

Radiace štruktúry

Jednoduchý príkaz **CASE**

- Jednoduchý *case* poskytuje podmienené vykonávanie na základe rovnosti operandov.
- *Search-expression* sa vyhodnotí (raz) a postupne sa porovnáva s každým výrazom v klauzulách *when*.
- Ak sa nájde zhoda, vykonajú sa zodpovedajúce príkazy a potom sa po *end case* riadenie odovzdá ďalšiemu príkazu.
- Syntax:

```
CASE search-expression
  WHEN expression [, expression [ ... ]] THEN
    statements
  [ WHEN expression [, expression [ ... ]] THEN
    statements
    ... ]
  [ ELSE
    statements ]
END CASE;
```

Riadiace štruktúry

Jednoduchý príkaz **CASE** - príklad

```
CASE x
  WHEN 1, 2 THEN
    msg := 'one or two';
  ELSE
    msg := 'other value than one or two';
END CASE;
```

Radiace štruktúry

Hľadaný príkaz **CASE**

- Hľadaná forma *case* poskytuje podmienené vykonávanie na základe pravdivosti booleovských výrazov.
- Booleovský výraz sa vyhodnocuje postupne pri každej klauzule *when*, až kým sa nenájde prvý, ktorý vráti hodnotu *true*.
- Následne sa vykonajú príslušné príkazy danej vetvy a potom sa po *end case* riadenie odovzdá ďalšiemu príkazu.
- Syntax:

```
CASE
  WHEN boolean-expression THEN
    statements
  [ WHEN boolean-expression THEN
    statements
    ... ]
  [ ELSE
    statements ]
END CASE;
```

Riadiace štruktúry

Hľadaný príkaz **CASE** - príklad

```
CASE
  WHEN x BETWEEN 0 AND 10 THEN
    msg := 'value is between zero and ten';
  WHEN x BETWEEN 11 AND 20 THEN
    msg := 'value is between eleven and twenty';
END CASE;
```

Riadiace štruktúry

Cyklus LOOP

- *Loop* definuje bezpodmienečnú slučku, ktorá sa opakuje donekonečna, kým nie je ukončená príkazom *exit* alebo *return*.
- Voliteľný *label* je možné použiť v príkazoch *exit* a *continue* vo vnorených cykloch na určenie, na ktorý cyklus sa tieto príkazy vzťahujú.
- Syntax:

```
[ <<label>> ]  
LOOP  
    statements  
END LOOP [ label ];
```


Riadiace štruktúry

Klauzula **EXIT**

- Syntax:
`EXIT [ label ] [ WHEN boolean-expression ];`
- Pokiaľ nie je uvedený *label*, ukončí sa najvnútornejší *loop* a vykoná sa nasledujúci príkaz po *end loop*.
- Ak je uvedený *label*, musí to byť *label* na aktuálnej alebo nejakej vonkajšej úrovni vnoreného *loop* alebo bloku.
- Ak je špecifikovaná klauzula *when*, ukončenie *loop* nastane, iba ak je podmienka pravdivá. V opačnom prípade tok riadenia prejde na príkaz po *exit*.
- *Exit* je možné použiť so všetkými typmi cyklov.

Riadiace štruktúry

Klauzula EXIT - príklad

```
LOOP
    -- some computations
    IF count > 0 THEN
        EXIT; -- exit loop
    END IF;
END LOOP;

LOOP
    -- some computations
    EXIT WHEN count > 0; -- same result as previous example
END LOOP;

<<ablock>>
BEGIN
    -- some computations
    IF stocks > 100000 THEN
        EXIT ablock; -- causes exit from the BEGIN block
    END IF;
    -- computations here will be skipped when stocks > 100000
END;
```

Riadiace štruktúry

Klauzula **CONTINUE**

- Syntax:
CONTINUE [**label**] [WHEN **boolean-expression**];
- Ak nie je uvedený *label*, začne sa ďalšia iterácia najvnútornejšieho *loop*.
- To znamená, že všetky príkazy zostávajúce v tele *loop* sú preskočené a tok riadenia sa vráti do výrazu riadenia *loop* (ak existuje), aby určil, či je potrebná ďalšia iterácia.
- Ak je *label* k dispozícii, určuje *label* cyklu, ktorého vykonávanie bude pokračovať.
- Ak je špecifikovaná klauzula *when*, ďalšia iterácia cyklu sa začne, iba ak je logický výraz pravdivý. V opačnom prípade tok riadenia prejde na príkaz po *continue*.
- *Continue* možno použiť so všetkými typmi cyklov.

Riadiace štruktúry

Cyklus **WHILE**

- Príkaz *while* opakuje postupnosť príkazov, pokiaľ sa boolovský výraz vyhodnocuje ako pravdivý.
- Výraz sa kontroluje pred každým vstupom do tela cyklu.
- Syntax:

```
[ <<label>> ]  
WHILE boolean-expression LOOP  
    statements  
END LOOP [ label ];
```

Riadiace štruktúry

Cyklus **WHILE** - príklad

```
do $$  
  
declare  
  
    counter integer := 0;  
  
begin  
  
    while counter < 5 loop  
  
        raise notice 'Counter %', counter;  
  
        counter := counter + 1;  
  
    end loop;  
  
end$$;
```

Riadiace štruktúry

Cyklus FOR (integer variant)

- Tento variant *for* vytvára cyklus, ktorý iteruje v rozsahu celočíselných hodnôt.
- Názov premennej je automaticky definovaný ako typ celé číslo a existuje iba vo vnútri cyklu (akákoľvek existujúca definícia názvu premennej je v rámci cyklu ignorovaná).
- Pri vstupe do slučky sa dva výrazy udávajúce dolnú a hornú hranicu rozsahu vyhodnotia jedenkrát.
- Ak klauzula *by* nie je zadaná, iteračný krok je 1.
- Ak je špecifikovaná klauzula *reverse*, potom sa po každej iterácii hodnota kroku odčíta.
- Syntax:

```
[ <<label>> ]  
FOR name IN [ REVERSE ] expression .. expression [ BY expression ] LOOP  
    statements  
END LOOP [ label ];
```

Radiace štruktúry

Cyklus **FOR** (integer variant) - príklad

```
FOR i IN 1..10 LOOP
    -- i will take on the values 1,2,3,4,5,6,7,8,9,10 within the loop
END LOOP;

FOR i IN REVERSE 10..1 LOOP
    -- i will take on the values 10,9,8,7,6,5,4,3,2,1 within the loop
END LOOP;

FOR i IN REVERSE 10..1 BY 2 LOOP
    -- i will take on the values 10,8,6,4,2 within the loop
END LOOP;
```

Riadiace štruktúry

Cyklus **FOR** (cez výsledok dopytu)

- Tento variant *for* cyklu umožňuje prechádzať výsledkami dopytu a zodpovedajúcim spôsobom s nimi manipulovať.
- *Target* je premenná typu záznam, *rowtype* alebo zoznam skalárnych premenných oddelených čiarkami.
- Do *target* sa postupne priraduje každý riadok vyplývajúci z dopytu a pre každý riadok sa vykoná telo cyklu.
- Syntax:

```
[ <<label>> ]  
FOR target IN query LOOP  
    statements  
END LOOP [ label ];
```


Riadiace štruktúry

Cyklus **FOR** (cez výsledok dopytu) - príklad

```
DO $$  
DECLARE  
    zaznammiest record;  
BEGIN  
    for zaznammiest in select nazov, populacia from mesto loop  
        raise notice 'Mesto % ma % obyvateľov.',  
            zaznammiest.nazov, zaznammiest.populacia;  
    end loop ;  
END  
$$;
```

Nabudúce

- Funkcie.
- Storované procedúry.
- Triggery.

Zdroje a použitá literatúra

- Jaroslav Porubän, Miroslav Biňas, Milan Nosál, *Databázové systémy*- prednášky, FEI, TUKE, 2011 - 2016.
- <https://www.postgresql.org/docs/>.
- <https://www.postgresqltutorial.com/>.
- Jim Mlodgenski, *Introduction to PL/pgSQL*, Amazon Web Services, 2019
- Ramez Elmasri, Shamkant B. Navathe: *Fundamentals of Database Systems*, Addison Wesley, 5 edition, 2006, 1168 p. ISBN 0321369572.
- Abraham Silberschatz, Henry F. Korth, S. Sudarshan: *Database System Concepts*, The McGraw-Hill Companies, 2011, 6th edition, ISBN 978-0-07-352332-3.
- Ярцев В.П. *Організація баз даних та знань*: навчальний посібник, Державний університет телекомунікацій, 214с, 2018.

Otázky?